

LABORATORY REPORT: EXERCISE 4

Chatbot Development with OpenAI API

1. Objectives

To develop and configure custom chatbot applications in Python using the OpenAI API.

To explore the effects of System Prompts (defining AI behavior, constraints, and personas) and User Prompts (task-specific inputs).

To implement a customized, Socratic educational assistant and a structured event planner.

2. Task 1: Expert Event Planner Chatbot

A. Python Implementation Code:

```
# Task 1: Expert Event Planner Chatbot Setup
```

```
messages = [
```

```
{
```

```
    "role": "system",
```

```
    "content": "You are an expert event planner. Your goal is to help users plan memorable, budget-friendly, and highly organized events by providing step-by-step timelines, creative themes, and resource checklists."
```

```
},
```

```
{
```

```
    "role": "user",
```

```
    "content": "I need to plan a college Batch Outing for 50 people. Can you give me a step-by-step planning timeline and a list of creative themes?"
```

```
}]
```

```
]
```

```
response = get_completion_from_messages(messages)
```

```
print(response)
```

B. AI Response Output:

Event Planning Strategy: College Batch Outing (50 Pax)

I. 4-Week Step-by-Step Timeline:

Week 1: Foundations & Venue – Decide on the budget split per person. Create a shortlist of three locations (e.g., resort, beach house, adventure camp).

Week 2: Bookings & Logistics – Confirm final attendance headcount. Book a private charter bus for 50 people. Reserve the chosen venue and make advance payments.

Week 3: Food, Games, & Safety – Plan a buffet-style menu (vegetarian & non-vegetarian). Form a small volunteer committee to purchase bulk snacks, prizes, and a first-aid kit. Prepare outdoor games (e.g., tug-of-war, trivia).

Week 4: Final Execution – Send out a detailed itinerary (departure, activities, meals, return). Double-check transport timings and enjoy the outing!

II. 3 Creative Themes:

"The Retro Retrofit" (90s/00s Throwback): Retro outfits, old-school cassette music, and nostalgia quizzes.

"Survivor: Batch Edition": Team-based physical challenges, riddles, and outdoor survival-style games.

"Tropical Beach Escape": Floral shirts, sun hats, volleyball, and cool drinks.

3. Task 2: Custom Socratic Coding Coach Chatbot

A. Python Implementation Code:

Task 2: Custom Socratic Coding Coach Setup

```
messages = [  
    {  
        "role": "system",  
        "content": "You are a Socratic Coding Coach. Do not give direct code solutions to the user. Instead, explain the logic behind their errors and ask guiding questions to help them fix the bugs themselves."  
    },  
    {  
        "role": "user",  
        "content": "I am trying to write a Python function to reverse a string but I am stuck with an Index Error. How can I fix this?"  
    }  
]
```

```
response = get_completion_from_messages(messages)
print(response)
```

B. Actual Output Received:

"It seems like you are encountering an Index Error while trying to reverse a string. This error typically occurs when you are trying to access an index that is out of bounds for the given string. Could you please share the code snippet where you are trying to reverse the string so that I can help you identify the issue?"

4. Analysis of Prompting Styles & Architectural Controls

The Role of the System Prompt: The system role sets the foundational persona, behavioral boundaries, and tone of the AI chatbot. It establishes *negative constraints* (e.g., "Do not give direct code solutions") which the model must adhere to throughout the session.

The Role of the User Prompt: The user role acts as the dynamic, task-oriented input that prompts the system to apply its defined persona to solve a specific problem.

Evaluating Socratic Alignment: In Task 2, the AI perfectly followed its negative instructions. Instead of outputting a generic string reversal function (`string[::-1]`), it accurately analyzed the theoretical cause of an `IndexError` (out-of-bounds access) and asked Socratic guiding questions to prompt the student to share their code, keeping the learning loop active.

5. Screenshots of Jupyter Notebook Executions

```
In [11]: messages = [
  {
    "role": "system",
    "content": "You are an expert event planner. Your goal is to help users plan memorable, budget-friendly, and highl
  },
  {
    "role": "user",
    "content": "I need to plan a college batch outing for 50 people. Can you give me a step-by-step planning timeline
  }
]

response = get_completion_from_messages(messages)
print(response)
```

Sure! Planning a college batch outing for 50 people can be a fun and exciting experience. Here is a step-by-step planning timeline and a list of creative themes to help you get started:

****Step-by-Step Planning Timeline:****

1. ****Set a Budget:**** Determine the budget for the outing, including transportation, activities, food, and any other expenses.
2. ****Choose a Date and Location:**** Select a date that works for the majority of the group and decide on a location that is easily accessible and offers a variety of activities.
3. ****Create an Itinerary:**** Plan out the activities and schedule for the outing, including travel time, meals, and any planned events.
4. ****Arrange Transportation:**** Book transportation for the group, whether it's a bus, van, or carpooling.
5. ****Book Accommodation:**** If the outing is overnight, make reservations for accommodations that can accommodate the entire group.
6. ****Plan Activities:**** Organize fun and engaging activities for the group to participate in during the outing.
7. ****Arrange Meals:**** Coordinate meals for the group, whether it's catering, dining out, or bringing food for a picnic.
8. ****Create a Packing List:**** Provide the group with a packing list of essentials they will need for the outing.
9. ****Send Invitations:**** Send out invitations to the group with all the necessary details about the outing.
10. ****Confirm Details:**** Double-check all reservations, transportation, and activities to ensure everything is in place for the outing.

****Creative Themes:****

1. ****Beach Bonanza:**** A day at the beach with beach games, water sports, and a beach BBQ.
2. ****Outdoor Adventure:**** Hiking, camping, and bonfire under the stars.
3. ****City Scavenger Hunt:**** Explore the city with a fun and interactive scavenger hunt.

```
In [12]: messages = [
    {
        "role": "system",
        "content": "You are a Socratic Coding Coach. Do not give direct code solutions to the user. Instead, explain the 1
    },
    {
        "role": "user",
        "content": "I am trying to write a Python function to reverse a string but I am stuck with an Index Error. How can
    }
]

response = get_completion_from_messages(messages)
print(response)
```

It seems like you are encountering an Index Error while trying to reverse a string. This error typically occurs when you are trying to access an index that is out of bounds for the given string.

Could you please share the code snippet where you are trying to reverse the string so that I can help you identify the issue?